

BDA Lab: Hadoop MapReduce Guide

Step-by-Step Instructions & Source Code

I. System Initialization

Before any experiment, start the Hadoop services in your terminal:

```
start-all.sh
```

Verify services are running:

```
jps
```

Create and enter your exam directory:

```
mkdir ~/bda_exam && cd ~/bda_exam
```

II. Experiment 1: Word Count

1. Create Java Files

Use nano Filename.java to create and paste the following:

WordCountMapper.java

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(Object key, Text value, Context context) throws IOException,
    InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}
```

WordCountReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) { sum += val.get(); }
        context.write(key, new IntWritable(sum));
    }
}
```

WordCountDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Word Count");
        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

2. Run Commands

```
nano input.txt # Add some text here
```

```
hdfs dfs -mkdir /input_wc
```

```
hdfs dfs -put input.txt /input_wc
```

```
javac -classpath `hadoop classpath` -d . WordCount*.java
```

```
jar -cvf wordcount.jar *.class
```

```
hadoop jar wordcount.jar WordCountDriver /input_wc /output_wc
```

```
hdfs dfs -cat /output_wc/part-r-00000
```

III. Experiment 2: Maximum Number

1. Source Code

MaxMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class MaxMapper extends Mapper<Object, Text, Text, IntWritable> {
    private Text word = new Text("max");
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        int number = Integer.parseInt(value.toString());
        context.write(word, new IntWritable(number));
    }
}
```

MaxReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MaxReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
        int max = Integer.MIN_VALUE;
        for (IntWritable val : values) {
            if (val.get() > max) { max = val.get(); }
        }
        context.write(key, new IntWritable(max));
    }
}
```

MaxDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
```

```
    Job job = Job.getInstance(conf, "Maximum Number");
    job.setJarByClass(MaxDriver.class);
    job.setMapperClass(MaxMapper.class);
    job.setReducerClass(MaxReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}
```

2. Run Commands

```
nano numbers.txt # Add numbers like 10, 50, 20
```

```
hdfs dfs -mkdir /input_max
```

```
hdfs dfs -put numbers.txt /input_max
```

```
javac -classpath `hadoop classpath` -d . Max*.java
```

```
jar -cvf max.jar *.class
```

```
hadoop jar max.jar MaxDriver /input_max /output_max
```

```
hdfs dfs -cat /output_max/part-r-00000
```

IV. Experiment 3: Filter Data (Map-Only)

1. Source Code

FilterMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class FilterMapper extends Mapper<Object, Text, NullWritable, Text> {
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        String[] fields = value.toString().split(",");
        if (fields[2].equals("HR")) {
            context.write(NullWritable.get(), value);
        }
    }
}
```

FilterDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class FilterDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Filter Data");
        job.setJarByClass(FilterDriver.class);
        job.setMapperClass(FilterMapper.class);
        job.setNumReduceTasks(0);
        job.setOutputKeyClass(NullWritable.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

2. Run Commands

```
nano employees.txt # Add lines like 1,John,HR
```

```
hdfs dfs -mkdir /input_filter
```

```
hdfs dfs -put employees.txt /input_filter
```

```
javac -classpath `hadoop classpath` -d . Filter*.java
```

```
jar -cvf filter.jar *.class
```

```
hadoop jar filter.jar FilterDriver /input_filter /output_filter
```

```
hdfs dfs -cat /output_filter/part-m-00000
```

V. Experiment 4: Average Calculation

1. Source Code

AvgMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class AvgMapper extends Mapper<Object, Text, Text, DoubleWritable> {
    private Text word = new Text("avg");
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        double num = Double.parseDouble(value.toString());
        context.write(word, new DoubleWritable(num));
    }
}
```

AvgReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class AvgReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws
        IOException, InterruptedException {
        double sum = 0; int count = 0;
        for (DoubleWritable val : values) {
            sum += val.get();
            count++;
        }
        context.write(key, new DoubleWritable(sum / count));
    }
}
```

AvgDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AvgDriver {
    public static void main(String[] args) throws Exception {
```



```

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Calculation");
    job.setJarByClass(AvgDriver.class);
    job.setMapperClass(AvgMapper.class);
    job.setReducerClass(AvgReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

2. Run Commands

```
nano decimals.txt # Add numbers like 10.5, 20.0
```

```
hdfs dfs -mkdir /input_avg
```

```
hdfs dfs -put decimals.txt /input_avg
```

```
javac -classpath `hadoop classpath` -d . Avg*.java
```

```
jar -cvf avg.jar *.class
```

```
hadoop jar avg.jar AvgDriver /input_avg /output_avg
```

```
hdfs dfs -cat /output_avg/part-r-00000
```

VI. Important Rules

- 1. Case Sensitivity:** Filenames MUST match the public class name exactly (e.g., `MaxReducer.java`).
- 2. Clean Output:** Hadoop will not overwrite. Delete output folders before re-running:

```
hdfs dfs -rm -r /your_output_folder
```